Technical Design Document (TDD)

System: Virtual Worker Platform (VWP)

Stack: n8n + Hugging Face + Postgres + Object Storage

---

1. Document Control

- **Document Title:** Technical Design Document (TDD)
- **System Name:** Virtual Worker Platform (VWP)
- **Version:** 0.1
- **Date:** 17/02/2026
- **Status:** Draft for build kickoff

---

2. Technical Goals

1. Deliver a secure multi-tenant platform with deterministic execution via n8n.
2. Implement a template-constrained AI interpretation layer using Hugging Face.
3. Enforce governance: RBAC, policy rules, approvals, and immutable audit.
4. Provide production-grade reliability: idempotency, retries, timeouts, observability.
5. Create a reusable foundation for adding templates/integrations with minimal code.

---

3. Physical Architecture & Deployment Topology

3.1 Environments

- **DEV:** developer testing; relaxed integrations; synthetic data
- **UAT:** stakeholder testing; representative integrations; test tenants
- **PROD:** strict governance; monitored; backups and DR enabled

3.2 Service Topology (Containerized)

Core Services

1. **vwp-api** (Backend)
2. REST API, auth, RBAC, task lifecycle, policy engine

3. Orchestrates HF inference and n8n execution

4. **vwp-ui** (Frontend)
5. Task intake, approvals, templates, dashboards
6. **n8n**
7. Workflow automation engine
8. Runs in **queue mode** for scale
9. **PostgreSQL**
10. Primary relational store
11. **Redis** (recommended)
12. n8n queue + optional backend job queue
13. **Object Storage** (S3 compatible)
14. Attachments & generated artifacts
15. **Observability Stack**
16. Logs (e.g., OpenSearch/ELK)
17. Metrics (Prometheus/Grafana)
18. Tracing (OpenTelemetry)
19. **Secrets / KMS**
20. Vault/KMS for backend secrets
21. n8n encrypted credential store for connector credentials

3.3 Network & Security Segmentation

- Public: **vwp-ui**, **vwp-api** (TLS)
- Private: Postgres, Redis, n8n internal endpoints
- Egress controls: only to approved integration domains
- WAF/rate limiting in front of vwp-api

---

4. Runtime Components

4.1 Backend (vwp-api)

Responsibilities

- Multi-tenant enforcement
- Auth + RBAC
- Task lifecycle state machine
- Template registry + schema validation
- AI gateway (HF calls)
- Policy engine (approval rules, rate limits)
- n8n bridge (execute, callback, idempotency)
- Audit/event emitter

Suggested Modules

- auth
- workspaces
- users
- templates
- tasks
- runs
- approvals
- policies
- ai
- audit
- notifications
- storage

4.2 AI Gateway (HF)

Interaction Model

- vwp-api calls HF endpoints to:
- classify
- extract entities to JSON
- summarize attachments (optional)

Hard Rules

- HF output must be **JSON only**
- Validate output via JSON Schema
- Store hashes of AI outputs for audit

4.3 Execution Engine (n8n)

Standard Workflow Contract

All n8n workflows must: - Accept a standard payload - Enforce idempotency - Call back to vwp-api for: - run started - approval request - run completed - run failed

---

5. Data Model (Logical → Physical)

5.1 Core Tables (Postgres)

workspaces

- id (uuid, pk)
- name
- status (active/suspended)
- created_at, updated_at

users

- id (uuid, pk)
- workspace_id (fk)
- email (unique per workspace)
- name
- status
- created_at

roles

- id (uuid, pk)
- name (admin/member/approver/viewer)

user_roles

- user_id (fk)
- workspace_id (fk)
- role_id (fk)
- primary key (user_id, workspace_id, role_id)

templates

- id (uuid, pk)
- workspace_id (fk) // allow per-tenant templates; optional global templates later
- name
- description
- risk_level (low/med/high)

- active_version_id (fk -> template_versions.id)
- created_by
- created_at, updated_at

template_versions

- id (uuid, pk)
- template_id (fk)
- version (int)
- input_schema_json (jsonb) // JSON Schema
- approval_policy_json (jsonb)
- allowed_actions_json (jsonb)
- n8n_workflow_id (text)
- n8n_workflow_version (text)
- changelog (text)
- created_by
- created_at

tasks

- id (uuid, pk)
- workspace_id (fk)
- created_by (fk -> users.id)
- title
- description
- priority (low/med/high)
- status (created/interpreting/needs_clarification/planned/awaiting_approval/running/completed/failed/cancelled)
- selected_template_id (fk)
- selected_template_version_id (fk)
- extracted_inputs (jsonb)
- risk_flags (jsonb)
- due_at (timestamp)
- created_at, updated_at

runs

- id (uuid, pk)
- task_id (fk)
- workspace_id (fk)
- template_version_id (fk)
- status (queued/running/paused/completed/failed)
- n8n_workflow_id
- n8n_execution_id
- idempotency_key (text, unique)
- attempt (int)
- started_at, finished_at
- outputs (jsonb)

- error_code (text)
- error_details (jsonb)

approvals

- id (uuid, pk)
- workspace_id (fk)
- task_id (fk)
- run_id (fk)
- scope (task|step)
- step_key (text, nullable)
- requested_by (fk -> users.id or service)
- requested_at
- status (pending/approved/rejected/expired)
- decided_by (fk -> users.id)
- decided_at
- decision_notes (text)

audit_events

- id (uuid, pk)
- workspace_id (fk)
- actor_type (user|service)
- actor_id (uuid, nullable)
- action (text)
- object_type (text)
- object_id (uuid)
- timestamp
- payload_redacted (jsonb)
- payload_hash (text)

attachments

- id (uuid, pk)
- workspace_id (fk)
- task_id (fk)
- filename
- content_type
- storage_url
- size_bytes
- sha256
- uploaded_by
- uploaded_at

artifacts

- id (uuid, pk)
- workspace_id (fk)
- run_id (fk)

- name
- type
- storage_url
- sha256
- created_at

5.2 Indexing & Tenant Safety

- Always index by **workspace_id** on high-volume tables (tasks, runs, audit_events, attachments)
- Composite indexes:
- tasks(workspace_id, status, created_at)
- runs(workspace_id, status, started_at)
- approvals(workspace_id, status, requested_at)
- Enforce row-level tenant scoping in application layer; optionally enable Postgres RLS later.

---

6. API Design (vwp-api)

6.1 API Principles

- Versioned: /api/v1/...
- Workspace scoping: derived from auth context (do not accept workspace_id from client unless admin switching workspaces)
- Validation on every boundary
- Idempotency for execution triggers
- Audit event emission for state changes

6.2 Core Endpoints (MVP)

Auth / Identity

- POST /auth/login
- GET /me

Tasks

- POST /tasks
- GET /tasks?status=&template=&q=
- GET /tasks/{taskId}
- POST /tasks/{taskId}/clarify (user supplies missing inputs)
- POST /tasks/{taskId}/cancel

Templates

- POST /templates
- GET /templates

- GET /templates/{templateId}
- POST /templates/{templateId}/versions
- POST /templates/{templateId}/activate-version/{versionId}

Runs

- POST /tasks/{taskId}/execute
- GET /runs/{runId}

Approvals

- GET /approvals?status=pending
- POST /approvals/{approvalId}/approve
- POST /approvals/{approvalId}/reject

Audit

- GET /audit?objectType=&objectId=&action=&from=&to=

n8n Callbacks (internal, authenticated by shared secret or mTLS)

- POST /internal/n8n/run-started
- POST /internal/n8n/approval-request
- POST /internal/n8n/run-completed
- POST /internal/n8n/run-failed
- POST /internal/n8n/resume (backend → n8n signal)

6.3 Standard Response Shapes

- Use consistent envelope:
- { success: true|false, data: ..., error: { code, message, details } }

---

7. Template Schema Specification

7.1 Template Input Schema (JSON Schema)

Each template version MUST store an input schema in JSON Schema Draft 2020-12 style.

Example (Email Send)

- recipient_email: string (format: email)
- subject: string (minLength 1)
- body: string (minLength 1)
- cc_list: array[email] (optional)
- attachments: array[attachment_id] (optional)

7.2 Validation Rules

- Reject execution if required fields missing
- Reject if format constraints fail
- Reject if policy constraints fail (e.g., external domain without approval)

7.3 Allowed Actions (Capability Constraints)

Store an allowlist to constrain n8n usage: - e.g., {"actions": ["send_email"], "systems": ["gmail"], "max_recipients": 10}

---

8. Policy Engine & Approval Matrix

8.1 Policy Evaluation Order

1. RBAC eligibility (who can execute)
2. Template allowed actions
3. Data validation (schema)
4. Risk scoring
5. Approval requirement
6. Rate limits / thresholds

8.2 Approval Triggers (MVP)

- External email domain → approval required
- CRM create/update/delete → approval required
- Delete operations → admin-only + approval
- Bulk operations > threshold → approval required

8.3 Risk Flags (Examples)

- external_domain
- bulk_operation
- pii_detected (optional, later)
- high_impact_system

---

9. n8n Workflow Standards

9.1 Standard Payload From Backend → n8n

Required: - workspace_id - task_id - run_id - template_id - template_version - inputs (json) - idempotency_key - callback_urls (complete, fail, approval_request)

9.2 Idempotency

- Each workflow must check idempotency_key before side effects.
- Persist step markers in vwp-api via internal endpoint:
- POST /internal/n8n/step-marker (optional in MVP)

9.3 Error Format

n8n must return structured error payload: - error_code - error_message - step - raw_provider_error (redacted)

9.4 Approval Pause/Resume

- n8n calls /internal/n8n/approval-request with:
- run_id, step_key, summary, risk_flags
- n8n waits for resume signal (webhook)

---

10. Security Design

10.1 Authentication

- MVP: email+password or magic link
- Enterprise option: SSO (OIDC/SAML)

10.2 Authorization

- RBAC enforced in vwp-api
- Approvals require role=approver or admin
- Template management restricted to admin

10.3 Tenant Isolation

- All DB reads/writes scoped by workspace_id
- Internal callbacks include workspace_id and are validated

10.4 Secrets Handling

- n8n credentials: encrypted and scoped per workspace
- Backend secrets in vault/KMS
- Never store OAuth tokens in task payloads

10.5 Data Protection

- TLS everywhere
- Encryption at rest (DB + object storage)
- Redaction rules for audit payloads

---

11. Observability

11.1 Logs

- Correlate using: workspace_id, task_id, run_id
- Emit structured logs (json)

11.2 Metrics

- task_created_count
- run_success_rate
- run_duration_seconds
- approvals_pending_count
- approval_turnaround_seconds
- ai_classification_latency
- ai_confidence_distribution

11.3 Tracing

- OpenTelemetry traces across vwp-api → HF → n8n callbacks

---

12. Reliability & Failure Handling

12.1 Retries

- Retries only for idempotent steps
- Exponential backoff
- Max attempts configurable per template

12.2 Timeouts

- Workflow execution timeout per template
- Approval timeout (expire)

12.3 Dead Letter / Manual Intervention

- Failed runs can be re-executed with same idempotency_key (guarded)
- Admin can mark as resolved with reason (audited)

---

13. Implementation Sequence (Engineering Plan)

Sprint 1: Platform Skeleton

- Auth + workspace model
- Task CRUD + state machine
- Basic UI for task submit and list

Sprint 2: Templates + AI Routing

- Template registry + versions
- HF classification + extraction
- Schema validation

Sprint 3: n8n Bridge + Execution Tracking

- Execute endpoint
- n8n callback endpoints
- Run tracking

Sprint 4: Approvals + Audit

- Approval creation and UI
- Pause/resume handshake
- Audit events and views

Sprint 5: Observability + Hardening

- Metrics/logs/traces
- Rate limiting
- Indexing and performance tuning

---

14. Deliverables

- TDD approved
- Postgres schema migration scripts
- API spec (OpenAPI)

- n8n workflow standards guide
- 3–5 MVP workflow templates implemented
- Security checklist + test evidence

END OF DOCUMENT